

Terminologie

INFO

Lesinhoud

In deze les ontdek je hoe je informatie kunt opslaan en verwerken in een script. Je leert wat variabelen zijn, hoe je ze benoemt en gebruikt, en hoe je met verschillende soorten gegevens (zoals tekst, getallen en booleans) werkt. We verkennen de basisinstructies (statements) die je nodig hebt om een script op te bouwen, en je krijgt inzicht in hoe JavaScript met data omgaat. Zo leg je een stevige basis voor het schrijven van echte code.

Doelstellingen

De student(e) kan:

- de basisprincipes van JavaScript-programmatie (variabelen en constanten, controlestructuren, functies, arrays, objecten, DOM en DOM-manipulatie, events en event handling) toepassen binnen een project.

▼ Inhoud

- [JavaScript](#)
- [De Console: jouw eerste contact met JavaScript](#)
 - [Hoe open je de console?](#)
 - [Eerste voorbeeld](#)
- [Commentaar](#)
 - [Single-line commentaar](#)
 - [Multi-line commentaar](#)

- [Variabelen en constanten](#)
 - [Metafoor: de doos](#)
 - [Keywords](#)
 - [Declareren, toekennen en initialiseren](#)
- [Statements in JavaScript](#)
 - [Wat doet een statement?](#)
- [Datatypes in JavaScript](#)
 - [Primitieve datatypes](#)
 - [undefined vs. null](#)
- [Operatoren in JavaScript](#)
 - [Toekeningsoperator](#)
 - [Rekenkundige operators](#)
- [Expressies in JavaScript](#)
 - [Expressies vs. Statements](#)
- [Scriptbestand linken aan een HTML-bestand](#)
 - [Mappenstructuur](#)
 - [De basis: het script onderaan de body](#)
 - [In de head met defer](#)
 - [Andere manieren](#)
- [Conclusie](#)

JavaScript

JavaScript is de *taal van de webbrowser*. Het is een programmeertaal die wordt gebruikt om webpagina's interactief en dynamisch te maken. Denk aan knoppen die reageren op een klik, formulieren die gegevens verwerken, of inhoud die verandert zonder dat de pagina opnieuw geladen wordt.

Met **JavaScript** kun je:

- de inhoud en de stijl van een pagina aanpassen terwijl die wordt bekeken
- reageren op acties van de gebruiker (zoals klikken, typen of scrollen);
- communiceren met servers om data op te halen of te versturen.

Om efficiënt te leren programmeren, is het belangrijk dat we allemaal dezelfde woorden gebruiken. Daarom besteden we aandacht aan de juiste **terminologie**: woorden zoals **variabele**, **statement**, **functie**, **object**, Deze termen helpen je om problemen helder te omschrijven en om oplossingen te formuleren die anderen (en jijzelf) kunnen begrijpen en verbeteren.

De Console: jouw eerste contact met JavaScript

Om kennis te maken met JavaScript, gebruiken we de **console**. Dit is een onderdeel van de ontwikkelaarstools (DevTools) die beschikbaar zijn in alle moderne webbrowsers. Via de console kun je JavaScript-code schrijven, uitvoeren en meteen zien wat er gebeurt. Ideaal dus om te experimenteren en te leren.

Hoe open je de console?

We gebruiken in dit opleidingsonderdeel allemaal **Google Chrome**. Volg deze stappen om de console te openen:

- **Windows**: druk op **F12** of **Ctrl + Shift + I**
- **Mac**: druk op **Cmd + Option + I**

Ga vervolgens naar het tabblad **Console**. Hier kun je JavaScript-code invoeren en uitvoeren.

Eerste voorbeeld

Alle voorbeelden kan je overnemen (of kopiëren) in de console om ze uit te proberen.

Typ of plak de volgende regel in de console en druk op Enter:

```
console.log("Hallo, wereld!");
```

js

Je ziet nu de tekst "Hallo, wereld!" verschijnen in de console, de waarde van hetgeen in `console.log()` staat. Daaronder zie je ook `undefined` staan en dat is normaal. Die lijn mag je voorlopig negeren. We focussen momenteel op wat `console.log()` toont: de waarde die we tussen de ronde haken meegeven. In dit geval dus de tekst "Hallo, wereld!".

CONVENTIE

In dit opleidingsonderdeel gebruiken we allemaal Google Chrome als browser.

Commentaar

Commentaar is tekst in de code die **niet wordt uitgevoerd** door JavaScript. Je gebruikt commentaar om uitleg te geven, je code leesbaar te houden, of om tijdelijk stukken code uit te schakelen.

In deze cursus gebruiken we commentaar om uitleg te geven over de code. Jij kan commentaar bij jouw oefeningen of projecten gebruiken voor:

- uit te leggen wat een stukje code doet
- notities toe te voegen voor jezelf of later voor je collega's

- je script overzichtelijk te houden

Single-line commentaar

Gebruik `//` om een enkele regel commentaar toe te voegen. Alles na `//` op dezelfde regel wordt genegeerd door JavaScript.

```
console.log("Hallo, wereld!"); // Dit is een  
commentaarregel  
  
// Dit is een andere commentaarregel  
// Dit is nog een andere commentaarregel
```

js

Multi-line commentaar

Gebruik `/*` en `*/` om commentaar over meerdere regels te schrijven. Alles tussen deze tekens wordt genegeerd door JavaScript.

```
/* Het kan gebruikt worden op één regel */  
/* Het kan ook gebruikt worden  
   over meerdere regels */  
  
/*  
 * Vaak wordt er gekozen om op elke regel een asterisk te  
plaatsen.  
 * Dit maakt het commentaar beter leesbaar.  
 */  
  
console.log("Hallo, wereld!"); // Dit is single-line  
commentaar
```

js

Variabelen en constanten

In JavaScript kun je **variabelen** en **constanten** gebruiken om gegevens op te slaan. Dit maakt het mogelijk om informatie te beheren en te manipuleren in je code.

Metafoor: de doos

METAFOOR

Een variabele is als een doos waarin je iets kunt stoppen. De naam van de variabele is het **label** op de doos, en de **waarde** van de variabele is wat in de doos zit.

```
const pi = 3.14; // Een doos met als label 'pi' en als  
waarde 3.14  
console.log(pi); // Toont de waarde van de constante 'pi'  
in de console
```

js

```
let naam = "John Duck"; // Een doos met als label 'naam'  
en als waarde 'John Duck'  
console.log(naam); // Toont de waarde van de variabele  
'naam' in de console
```

js

Keywords

In JavaScript zijn er bepaalde **keywords** (Nederlands: sleutelwoorden) die speciale betekenissen hebben zoals `var`, `let` en `const`. Deze woorden kunnen niet worden gebruikt als namen voor variabelen (of functies), omdat ze al een specifieke betekenis hebben in de taal.

Wanneer je informatie opzoekt rondom JavaScript, zal je nog regelmatig `var` tegenkomen. Dit is een keyword dat vroeger werd gebruikt om variabelen te *declareren*.

Bijvoorbeeld:

```
var leeftijd = 25; // Een doos met als label 'leeftijd'
                en als waarde 25
console.log(leeftijd); // Toont de waarde van de
variabele 'leeftijd' in de console
```

js

Maar in moderne JavaScript gebruiken we `let` en `const`.

CONVENTIE

- Gebruik `let` voor **variabelen** die kunnen veranderen, ze zijn *variabel*. Iemands naam kan altijd veranderd worden, is dus variabel waardoor we `let` gebruiken.
- Gebruik `const` voor variabelen - **constanten** die niet kunnen veranderen, ze zijn *constant*. De wiskundige waarde *pi* verandert niet, is dus constant waardoor we `const` gebruiken.
- Gebruik **geen** `var`

Naamgeving van variabelen

De naam van een variabele heeft enkele regels:

- De naam moet beginnen met een **letter**, een **dollar-teken** `$` of een **liggend streepje** `_`.
- De naam mag **geen spaties** bevatten.
- De naam mag **geen speciale tekens** bevatten (behalve het dollar-teken `$` en het liggend streepje `_`).

CONVENTIE

Binnen dit opleidingsonderdeel gebruiken we `$` en `_` **niet** in de namen van variabelen.

Binnen dit opleidingsonderdeel gebruiken we `camelCase` voor de naamgeving van variabelen:

- De eerste letter is klein
- Elk volgend woord begint met een hoofdletter. Bijvoorbeeld:

```
let voornaam;  
let totaalBedrag;  
let aantalPersonen;
```

js

LET OP

Namen van variabelen zijn hoofdlettergevoelig. `voornaam`, `Voornaam` en `VOORNAAM` zijn drie verschillende variabelen.

Declareren, toekennen en initialiseren

- **Declareren**: je maakt een variabele (`let` of `const`) aan, maar geeft nog geen waarde. Bijvoorbeeld `let getal;`
- **Toekennen**: je geeft een waarde **aan een bestaande variabele**. Bijvoorbeeld `getal = 10;`
- **Initialiseren**: is het declareren en toekennen van een waarde aan een variabele in één stap. Bijvoorbeeld `let getal = 10;`

METAFOOR

- **Declareren** = het klaarzetten van een doos met een label erop, zonder al iets in de doos te stoppen.
- **Toekennen** = iets in de doos te stoppen nadat je de doos hebt klaargezet.

- **Initialiseren** = het klaarzetten van de doos en er meteen iets in stoppen.

```
let voornaam; // Declareren van een variabele met de naam
'voornaam', zonder waarde
voornaam = "John"; // Toekennen van de waarde 'John' aan
de variabele 'voornaam'
let achternaam = "Duck"; // Initialiseren van de
variabele met de naam 'achternaam' en de waarde 'Duck'
const geboortedatum = "13-08-1991"; // Initialiseren van
de constante variabele met de naam 'geboortedatum' en de
waarde '13-08-1991'
```

▼ Oefening: zelf aan de slag

Probeer deze stappen in de console:

```
let stad; // Declareren
stad = "Hasselt"; // Toekennen
console.log(stad); // Toont 'Hasselt'

const land = "België"; // Initialiseren
console.log(land); // Toont 'België'
```

Statements in JavaScript

Een **statement** is een instructie die door JavaScript wordt uitgevoerd. Het is een bouwsteen van je script: elke regel die iets doet, is een statement.

Voorbeelden van statements:

js

```
let leeftijd; // Statement om een variabele te declareren  
leeftijd = 25; // Statement om een waarde toe te kennen  
aan de variabele 'leeftijd'
```

```
const pi = 3.14; // Statement om een constante te  
declareren en een waarde toe te kennen (te initialiseren)
```

```
console.log(leeftijd); // Statement om de waarde van  
'leeftijd' naar de console te loggen
```

Wat doet een statement?

Een statement vertelt de browser: *Voer deze actie uit.*

Dat kan bijvoorbeeld zijn:

- een variabele aanmaken
- een waarde toekennen
- iets tonen in de console
- een berekening uitvoeren
- een beslissing nemen (later via if-statements)

CONVENTIE

De puntkomma `;` aan het einde van een statement is optioneel in JavaScript, maar het is goede gewoonte om die expliciet te gebruiken. Zo wordt duidelijk waar een statement eindigt, zeker als je meerdere statements op één regel zou schrijven (wat we voorlopig vermijden).

In dit opleidingsonderdeel zullen we **de puntkomma altijd gebruiken**.

▼ Oefening: statements herkennen

Voer deze code uit in de console en beantwoord de vragen:

```
let voornaam = "John";
let achternaam = "Duck";
let volledigeNaam = voornaam + " " + achternaam;

console.log(volledigeNaam);
```

js

Vragen:

- Hoeveel statements zie je in deze code?
- Wat doet elk statement?
- Wat gebeurt er als je een puntkomma weglaat?

Datatypes in JavaScript

In JavaScript bepaalt een datatype welk soort waarde een variabele bevat. Datatypes zijn belangrijk omdat ze bepalen hoe je met die waarde kunt werken.

Primitieve datatypes

Voorlopig maken we kennis met de **primitive datatypes** (Nederlands: primitieve/simpele datatypes), de eenvoudige bouwstenen van JavaScript:

Datatype	Beschrijving	Voorbeeld
string	Een reeks tekens (tekst)	"Hallo, wereld!"
number	Een getal (geheel of decimaal)	42 of 3.14
boolean	Een waarde die waar of niet waar kan zijn	true of false

Datatype	Beschrijving	Voorbeeld
undefined	Een variabele die is gedeclareerd, maar nog geen waarde heeft gekregen	<code>let x;</code>
null	Een expliciete lege waarde, die aangeeft dat er geen waarde is.	<code>let y = null</code>

```

let naam = "John Duck"; // string
let leeftijd = 25; // number
let isStudent = true; // boolean
let x; // undefined
let y = null; // null

```

js

undefined vs. null

METAFOOR

- `undefined` betekent: *"Er is een doos met een label, maar er zit nog niets in."*
- `null` betekent: *"Er is een doos, en we hebben er bewust niets in gestopt."*

CONVENTIE

Als developer gebruiken we `null` om aan te geven dat een variabele geen waarde heeft. Terwijl `undefined` aangeeft dat een variabele is gedeclareerd, maar nog geen waarde heeft toegekend gekregen.

In dit opleidingsonderdeel zullen we nooit `undefined` manueel toekennen - om zo onderscheid te maken tussen een variabele die nog geen waarde heeft gekregen en een variabele die expliciet geen waarde heeft.

EXTRA (optioneel)

Er zijn nog twee andere primitive datatypes in JavaScript:

- **bigint**: Een datatype voor zeer grote gehele getallen, bijvoorbeeld `1234567890123456789012345678901234567890n`.
- **symbol**: Een datatype voor unieke en onveranderlijke waarden, bijvoorbeeld `Symbol("beschrijving")`.

▼ Oefening: herken het datatype

Voer deze code uit in de console en probeer te benoemen welk datatype elke variabele heeft:

```
let titel = "Digitale Vormgeving";  
let aantalStudenten = 18;  
let isActief = true;  
let locatie;  
let opmerking = null;
```

js

Vragen:

- Welk datatype heeft elke variabele?
- Wat zie je als je `console.log(typeof locatie);` uitvoert?
- Wat is het verschil tussen `undefined` en `null` in deze context?

Operatoren in JavaScript

Een **operator** is een symbool dat een bewerking uitvoert op één of meerdere waarden. Je hebt al kennismemaakt met de **toekenningoperator** `=` in de voorbeelden hierboven, die een waarde toekent aan een variabele.

Later in de cursus leer je ook andere soorten operatoren kennen, zoals **vergelijkingsoperatoren** en **logische operatoren**. Voor nu

focussen we op de **toekenning** en de **rekenkundige operatoren**.

Type operator	Voorbeelden	Doel
Toekenningoperator	=	Waarde toekennen aan een variabele
Rekenkundige operators	+, -, *, /, %	Wiskundige bewerkingen uitvoeren
Vergelijkingsoperators	==, ===, !=, !==, <, >, <=, >=	Waarden vergelijken
Logische operators	&&, , !	Logische bewerkingen uitvoeren

Toekenningsoperator

De toekenningoperator, **=**, **wijst een waarde toe** aan een variabele.

```
const getal = 5; // De waarde 5 van het type number wordt  
toegekend aan de constante variabele genaamd 'getal'  
  
let tekst = "Hallo, wereld!"; // De waarde 'Hallo,  
wereld!' van het type string wordt toegekend aan de  
variabele genaamd 'tekst'
```

js

Rekenkundige operators

Rekenkundige operators worden gebruikt om wiskundige bewerkingen uit te voeren. Een opfrisser van de basis:

js

```
const getal1 = 5;
const getal2 = 10;

const som = getal1 + getal2; // Optelling
const verschil = getal2 - getal1; // Aftrekking
const product = getal1 * getal2; // Vermenigvuldiging
const quotient = getal2 / getal1; // Deling
const rest = getal2 % getal1; // Modulo (rest van de deling)

console.log(som); // Toont 15
console.log(verschil); // Toont 5
console.log(product); // Toont 50
console.log(quotient); // Toont 2
console.log(rest); // Toont 0
```

Denk ook aan de **prioriteit** van bewerkingen. JavaScript volgt **wiskundige regels** voor de volgorde van bewerkingen:

- Haakjes `()`
- Vermenigvuldiging `*` en deling `/`
- Optelling `+` en aftrekking `-`

Bij gelijke prioriteit worden de bewerkingen van **links naar rechts** uitgevoerd.

js

```
const resultaat = 5 + 3 * 2; // Resultaat is 11, omdat 3
* 2 = 6 en 5 + 6 = 11
console.log(resultaat); // Toont 11

const resultaat2 = (5 + 3) * 2; // Resultaat is 16, omdat
de haakjes eerst worden uitgevoerd, dus 5 + 3 = 8 en 8 *
2 = 16
console.log(resultaat2); // Toont 16

const resultaat3 = 10 / 2 - 1; // Resultaat is 4, omdat
```

```
10 / 2 = 5 en 5 - 1 = 4
console.log(resultaat3); // Toont 4

const resultaat4 = 10 % 3; // Resultaat is 1, omdat 10
gedeeld door 3 een rest van 1 heeft (3 past 3 keer
volledig in 10 er blijft 1 over)
console.log(resultaat4); // Toont 1
```

▼ Oefening: experimenteer met operatoren

Voer deze code uit in de console en probeer zelf variaties:

```
let a = 8;
let b = 3;

console.log(a + b); // Optelling
console.log(a - b); // Aftrekking
console.log(a * b); // Vermenigvuldiging
console.log(a / b); // Deling
console.log(a % b); // Modulo

let resultaat = (a + b) * 2;
console.log(resultaat); // Wat gebeurt er?
```

js

Expressies in JavaScript

Een **expressie** is een stukje code dat evalueert naar één enkele waarde. Die waarde kan een getal, een string, een boolean, een object, enzovoort zijn. Je hebt al enkele expressies gezien in de voorbeelden hierboven.

Enkele voorbeelden van hierboven:

```
const getal = 5;
// De expressie '5' evalueert naar de waarde 5, die wordt
toegekend aan de constante variabele genaamd 'getal'.
```

js

```
const resultaat = getal * 2;  
// De expressie 'getal * 2' evalueert naar een enkele  
waarde, die wordt toegekend aan de constante variabele  
genaamd 'resultaat'.  
  
const som = 5 + 3 * 2;  
// De expressie '5 + 3 * 2' evalueert naar de waarde 11,  
die wordt toegekend aan de constante variabele genaamd  
'som'.
```

Expressies vs. Statements

- **Expressie:** levert een waarde op. Voorbeeld: `5 + 3`, `"Hallo" + "wereld"`, `x * 2`.
- **Statement:** voert een actie uit, maar levert niet direct een waarde op. Hier komen we later nog op terug bij onder andere *Conditionele Statements*

▼ Oefening

Welke van de volgende zijn expressies?

- `10 + 20`
- `const x = 5;`
- `"Hallo" + " wereld"`

Scriptbestand linken aan een HTML-bestand

Je leert hoe je een extern JavaScript-bestand (bijv. script.js) correct laadt in een HTML-pagina, waar je het `<script>-element` plaatst, en wat het

verschil is tussen `defer`, `async` en `type="module"`.

Mappenstructuur

Een duidelijke structuur helpt fouten vermijden.

```
project/  
├ index.html  
└ js/  
  └ script.js
```

js

▼ oefening

Maak een nieuw project aan en bouw de bovenstaande structuur na.

Zorg voor een basis HTML-structuur in het `index.html`-bestand.

We gaan `script.js` laden in `index.html`.

De basis: het script onderaan de body

De eenvoudigste en veilige manier is het script toevoegen onderaan in het `<body>`-element. Alle HTML-code is dan namelijk al ingeladen en je script kan meteen elementen aanspreken.

```
<!doctype html>  
<html lang="nl">  
  <head>  
    <meta charset="utf-8" />  
    <title>Mijn eerste script</title>  
  </head>  
  <body>  
    <h1>Hallo!</h1>
```

html

```
<!-- Laad het script net voor de sluitende body-tag -
->
<script src="/js/script.js"></script>
</body>
</html>
```

▼ oefening

Voeg het script toe aan het `index.html` bestand.

Schrijf in het JavaScript-bestand de volgende code:

```
console.log("Script geladen!");
```

js

Open het `index.html` -bestand in de browser en test jouw code.

In de head met defer

Wil je scripts in de `<head>` houden, gebruik dan `defer`. `defer` laadt het script *asynchroon*, maar wacht met uitvoeren tot de HTML klaar is.

```
<!doctype html>
<html lang="nl">
  <head>
    <meta charset="utf-8" />
    <title>Mijn eerste script</title>
    <!-- Laad het script in de head in -->
    <script src="/js/script.js" defer></script>
  </head>
  <body>
    <h1>Hallo!</h1>
  </body>
</html>
```

html

CONVENTIE

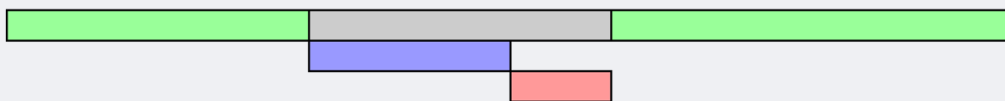
Wij laden onze scripts in deze cursus **altijd** in op deze manier. Dus met

`defer` in de `<head>`

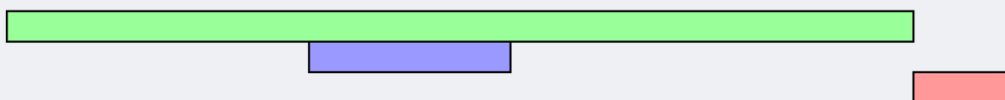
▼ Waarom `defer` ?

Om te visualiseren waarom we hiervoor kiezen, kijken we naar onderstaande afbeeldingen:

Wanneer we het script in de body plaatsen, dan wordt de HTML ingeladen (groene kleur), zodra JavaScript een `<script>` -element tegenkomt, stopt het met het inladen van de HTML (grijze kleur) tot het script is ingeladen (paarse kleur) en uitgevoerd (rode kleur). Pas daarna gaat het verder met het inladen van de HTML (groene kleur).



Wanneer we het script in de head plaatsen met `defer`, dan wordt de HTML ingeladen (groene kleur), zodra JavaScript een `<script>` -element tegenkomt, laadt het het script asynchroon (paarse kleur) en gaat ondertussen verder met het inladen van de HTML (groene kleur). Zodra de HTML volledig is ingeladen (groene kleur), wordt het script uitgevoerd (rode kleur).



Dit is efficiënter (HTML en JavaScript wordt tegelijkertijd ingeladen) en voorkomt fouten omdat je script altijd kan rekenen op een volledig ingeladen HTML voor het wordt uitgevoerd.

▼ oefening

Wijzig jouw code in het `index.html` bestand zodat jouw script met `defer` wordt ingeladen.

Open het `index.html` bestand in de browser en test jouw code

Andere manieren

Er zijn nog 2 andere manieren waarop het script kan ingeladen worden, maar hier gaan we tijdens dit opleidingsonderdeel niet verder op in. Deze manier zijn:

- In plaats van `defer`, `async` gebruiken. Dit laadt en voert het script uit zodra het binnen is, onafhankelijk van de HTML of andere scripts. Dit wordt voornamelijk gebruikt voor scripts die niet afhankelijk zijn van de HTML elementen.
- **Modules**: met ES Modules kan je code opdelen en importeren/exporteren. **Modules zijn van nature "deferred"**.

Conclusie

Door de basisprincipes van JavaScript te begrijpen, leg je een sterke fundering voor al je toekomstige codeerprojecten. Door dezelfde terminologie te gebruiken, communiceren we duidelijk en vermijden we misverstanden.

We hebben kennisgemaakt met variabelen, constanten, statements, datatypes, operators en expressies – de bouwstenen van JavaScript.

Met deze basis ben je klaar om je eerste oefeningen te maken en je programmeervaardigheden verder te ontwikkelen!

Previous page
[Scratch](#)

Next page
[Oefeningen](#)

